

ACNC Registry data: Arrow version

Ian D. Gow

1 October 2024

This code shows how one can use **list columns** (e.g., in a parquet file) to provide a single-file (or single-table) representation of data that might naturally be stored as multiple tables in a more traditional **relational database**. The code to produce the parquet file used in the following analysis is provided [here](#).

In the original registry data supplied by the ACNC, the data I have stored in list columns were spread over multiple columns. For example, “Operating locations (columns R-Z)” included columns such as “Operates in ACT” and “Operates in VIC” with values equal to either **Y** or blank. I converted these columns to a single column, **states**, with values such as **VIC** or **VIC, NSW**. While these look like simply comma-separated text values when viewing the data in software such as [Tab](#), they are actually list columns.

Other list columns include **operating_countries** (originally a single column, but as comma-separated text, not a list column), **subtypes** (originally “Subtypes (columns AA-AN)”), and **beneficiaries** (originally “Beneficiaries (columns AO-BN)”). Below I provide examples of working with such columns.

In writing this note, I use the packages listed below.¹ This note was written using [Quarto](#) and compiled with [RStudio](#), an integrated development environment (IDE) for working with R. The source code for this note is available [here](#) and the latest version of this PDF is [here](#).

```
library(tidyverse)
library(tinytable)
library(arrow)
library(farr)
```

We start by downloading the data, which takes a few seconds.

```
registry <-
  read_parquet('https://go.unimelb.edu.au/5d78') |>
  collect() |>
  system_time()
```

```
user  system elapsed
0.446  0.056   4.312
```

We can construct the **beneficiaries** data frame by using **unnest()** with the list column **beneficiaries**.

```
beneficiaries <-
  registry |>
  select(abn, beneficiaries) |>
```

¹Execute `install.packages(c("tidyverse", "arrow", "tinytable", "farr"))` within R to install all the packages you need to run the code in this note.

```
unnest(beneficiaries) |>  
rename(beneficiary = beneficiaries)
```

Charities vary in terms of the groups they serve, or **beneficiaries**. The results of the following code are shown in Table 1.

```
registry |>  
  unnest(beneficiaries) |>  
  count(beneficiaries, sort = TRUE) |>  
  tt() |>  
  style_tt(aligned = "ld") |>  
  format_tt(escape = TRUE)
```

Table 1: Number of charities serving each beneficiary type

beneficiaries	n
Youth	24633
Adults	23992
Families	23188
General Community in Australia	22638
Children	22130
Aged Persons	21763
Females	19027
Males	18012
Financially Disadvantaged	15826
Early Childhood	15184
Rural Regional Remote Communities	14757
Ethnic Groups	14384
Aboriginal or TSI	13528
People with Disabilities	13396
People at risk of homelessness	9493
Unemployed Person	9327
People with Chronic Illness	8082
Other Charities	6513
Veterans or their families	5656
Victims of crime	5220
Victims of Disasters	4856
Communities Overseas	4710
Migrants Refugees or Asylum Seekers	3735
Pre Post Release Offenders	3612
Gay Lesbian Bisexual	2890

Many charities serve multiple beneficiary types. The most common pairs of beneficiary types are given in Table 2, which is produced using the following code.

```
beneficiaries |>
  inner_join(beneficiaries, by = "abn",
             relationship = "many-to-many") |>
  filter(beneficiary.x < beneficiary.y) |>
  count(beneficiary.x, beneficiary.y) |>
  arrange(desc(n)) |>
  head(n = 10) |>
  tt() |>
```

```
style_tt(align = "lld") |>
format_tt(escape = TRUE)
```

Table 2: Most common beneficiary pairs

beneficiary.x	beneficiary.y	n
Adults	Aged Persons	19277
Adults	Youth	18429
Children	Youth	17384
Females	Males	17182
Adults	Families	16247
Aged Persons	Youth	15787
Families	Youth	15381
Aged Persons	Families	15048
Adults	Females	14088
Children	Families	13987

The results of the following code are shown in Table 3.

```
registry |>
  unnest(operating_countries) |>
  select(abn, operating_countries) |>
  filter(operating_countries != "AUS") |>
  count(operating_countries, sort = TRUE) |>
  head(n = 10) |>
  tt() |>
  format_tt(escape = TRUE)
```

Table 3: Most common countries of operation

operating_countries	n
IDN	430
PHL	385
PNG	371
KEN	360
UGA	299
NPL	270
FJI	263
IND	247
THA	241
VNM	240

The results of the following code are shown in Table 4.

```

registry |>
  unnest(operating_countries) |>
  distinct(abn, operating_countries) |>
  filter(operating_countries != "AUS") |>
  count(abn, name = "num_countries", sort = TRUE) |>
  mutate(num_countries = if_else(num_countries > 10, "More than 10",
                                as.character(num_countries)),
         num_countries = fct_inorder(num_countries)) |>
  count(num_countries) |>
  arrange(desc(num_countries)) |>
  tt() |>
  style_tt(aligned = "ld") |>
  format_tt(escape = TRUE)

```

Table 4: Number of countries of operation per charity

num_countries	n
1	1711
2	455
3	237
4	137
5	112
6	79
7	56
8	42
9	35
10	29
More than 10	187

The results of the following code are shown in Table 5.

```

registry |>
  unnest(subtypes) |>
  count(subtypes, sort = TRUE) |>
  head(n = 10) |>
  tt() |>
  style_tt(aligned = "ld") |>
  format_tt(escape = TRUE)

```

Table 5: Most common charity subtypes

subtypes	n
Advancing Religion	16954
Advancing social or public welfare	12624
Advancing Education	11887
PBI	11696
Purposes beneficial to ther general public and other analogous	6674
Advancing Health	6305
Advancing Culture	5121
HPC	2463
Advancing natual environment	2153
Promoting reconciliation mutual respect and tolerance	1440